Faculty of Engineering & Information Technology
University of Technology, Sydney

31338 Network Servers
Lab 10c: Virtual hosting with Apache

Aims

1. To configure a Linux machine to act as a web server for multiple DNS domains

Task 1: Virtual hosting a second domain

A common requirement in organisations is to have a web server answer requests for more than one domain name. For example, the company's main domain name might be `www.redhat.com`, but the company might also want a web site named after one of their products, e.g., `www.fedoraproject.org`. Or the company might have two different divisions, e.g. `www.telstra.com` and `www.bigpond.net.au`. Although the two different domains could be hosted on two separate computers, often organisations prefer to host multiple web domains on the same computer, to save resources. To enable this, you need to set up virtual hosting.

Virtual hosting can be implemented in three different ways, IP-based, name-based and dynamic. In the following exercise, we set up name-based virtual hosts. Name-based virtual hosting uses a feature introduced in HTTP version 1.1 to distinguish the target website, even though these websites reside on the same machine and are reached by browsers using identical IP addresses. A client using HTTP/1.1 sends a Host header to identify the hostname of the server that should respond to the request. Apache works out which page to serve based on the contents of the Host header which arrived with the request.

For this exercise, we need a second domain name which can be resolved to an IP address. We can implement this using DNS as follows. Create a second domain name in the DNS server (you will need a separate entry in the `named.conf` file, and you will need to create a separate DNS zone file). An easier alternative, which is sufficient to prove your implementation of virtual hosts, is to define the two or more web server fully qualified domain names in the /etc/hosts file of the VM that you will use to send HTTP requests to the VM running Apache.

Change your Apache configuration file so that it will also accept web requests for the second (virtual) domain. The config entries for virtual hosting are at the end of the `httpd.conf` file. Specify a different *DocumentRoot* for the virtual host (create a new directory, and put a new `index.html` file inside it for the virtual host).

You will need to turn on the *NameVirtualHost* directive in the `httpd.conf` file, and you will need to have `<VirtualHost>` entries for **both** your main hostname and your virtual hostname.

Test to verify that web requests to the original host, use Firefox to retrieve one web page (from your original *DocumentRoot*), while web requests to the second (virtual) host retrieve a different web page (from a different *DocumentRoot*).

Refer to the example on virtual hosting overleaf. This can be used as a guide for your configuration. This example was taken from the Apache website. The full URL is

```
http://httpd.apache.org/docs/current/vhosts/examples.html
```

The above website has examples of configurations covering a number of other scenarios. Interested students will be able to uncover many other websites and examples for configuring Apache.

Task 2: Testing from your host operating system

As an added test, try connecting from your host machine (not the VM). To do this you need to know the IP address configured on *ens33*, e.g. probably 192.168.3.2 if you are in the UTS labs, otherwise use `ifconfig` or `nmcli`. Why doesn't it connect? (Hint: use `firewall-config` to add `http` and `https` into the *public* zone).

If you have superuser/Administrator privileges on your host machine (i.e. you are NOT working in the UTS labs), add an entry into your "hosts" file for your virtual servers, mapped to the *ens33* IP address of your virtual machine. E.g. on

Linux, edit `/etc/hosts`. On Windows, edit `C:\Windows\System32\drivers\etc\hosts` (on Windows, when opening Notepad you may need to "Run as administrator"). In either case, the line to add will look like the one below, but change the IP address and names to match yours:

```
192.168.3.2    www.it.netserv.edu.au   www2.it.netserv.edu.au
```

Now from your host OS you should be able to type those hostnames in your web browser, and see the different virtual host sites. Remember to delete the line above from your hosts file on your host OS when you are done testing.

**** END OF LAB TASK ****

Below is for reference – taken from: `http://httpd.apache.org/docs/current/vhosts/examples.html`

Running several name-based web sites on a single IP address.

Your server has multiple hostnames that resolve to a single address, and you want to respond differently for www.example.com and **www.example.org**.

Note:
Creating virtual host configurations on your Apache server does not magically cause DNS entries to be created for those host names. You must have the names in DNS, resolving to your IP address, or nobody else will be able to see your web site. You can put entries in your hosts file for local testing, but that will work only from the machine with those hosts entries.

Server configuration

```
# Ensure that Apache listens on port
80 Listen 80
<VirtualHost *:80>
   DocumentRoot "/www/example1"
   ServerName www.example.com

   Other directives here
</VirtualHost>

<VirtualHost *:80>
    DocumentRoot "/www/example2"
    ServerName www.example.org

    Other directives here
 </VirtualHost>
```

The asterisks match all addresses, so the main server serves no requests. Due to the fact that the virtual host with `ServerName www.example.com` is first in the configuration file, it has the highest priority and can be seen as the *default* or *primary* server. That means that if a request is received that does not match one of the specified `ServerName` directives, it will be served by this first `<VirtualHost>`.

Note:
You may replace `*` with a specific IP address on the system. Such virtual hosts will only be used for HTTP requests received on connection to the specified IP address.

```
NameVirtualHost 172.20.30.40

<VirtualHost 172.20.30.40>
# etc ...
```

However, it is additionally useful to use `*` on systems where the IP address is not predictable - for example if you have a dynamic IP address with your ISP, and you are using some variety of dynamic DNS solution. Since `*` matches any IP address, this configuration would work without changes whenever your IP address changes.

The above configuration is what you will want to use in almost all name-based virtual hosting situations. The only thing that this configuration will not work for, in fact, is when you are serving different content based on differing IP addresses or ports.